

Categorical Framework for Visualization Software Design

Category: Research

ABSTRACT

The contract data visualization tools make with their users is that a chart is a faithful and accurate visual representation of the numbers it is made from. Motivated by wanting to make tools that are more flexible while still rigorously structure preserving, we propose a methodology for fully specifying arbitrary data to visualization mappings in a manner that translates to code. We propose that fiber bundles provide a uniform interface for describing a variety of underlying data - tables, images, networks, etc. - in a manner that independently encodes the mathematical structure of the topology and the fields of the dataset. Modeling the data structures that store the datasets as sheaves provide a method for specifying visualization methods that are designed to work regardless of how the dataset is stored - whether the data is on disk, distributed, or on demand. Specifying the visualization library components as natural transforms of sheaves means that the constraints that the component must satisfy to be structure preserving can be specified as the set of morphisms on the data and graphic sheaves, including the structure on the topology and fields of the data. Using category theory to formally express how visual elements are constructed means we can translate those expectations into code, which can then be used to enforce the expectation that a visualization tool is faithfully translating between numbers and charts.

Index Terms: Visualization software design, visualization theory, categorical framework

1 INTRODUCTION

Visualization design guidelines, generally, describe how to choose visual encodings that preserve the structure of the data; to follow these guidelines the visualization tools that implement these data $\rightarrow$ graphic transforms must be structure preserving. Loosely, preserving structure means that the properties of the data and how the points are connected to each other should be inferable from the graphic such that a graphic $\rightarrow$ data mapping can be made. For example, values read off a bar chart have to be equivalent to the values used to construct that chart. Therefore a visualization tool is structure preserving when it preserves the bidirectional mapping data $\leftrightarrow$ graphic.

We propose that structures from category theory and algebraic topology can be used as a sort of pseudo-code for uniformly specifying the behaviors and properties of data and graphic and rigorously expressing the structure preserving constraints on maps between them. For example, our framework can encapsulate how a table and scatter plot and heatmap are different representations of the same data and track an observation from a data cube as a point along a time series and on a map and in a network. By design, this pseudo-code translates to strong programmatic types and a functional design framework, which enable visualization software developers to build complex components from simpler verifiable parts [27, 28]. These components can be built as a standalone library and integrated into existing libraries and we hope these ideas will influence the architecture of critical data visualization libraries, such as Matplotlib.

The contribution of this paper is a methodology for describing structure, verifying structure preservation, and specifying the conditions for constructing a structure preserving map between data and graphics. This framework also provides guidance for the construction and testing of structure preserving visualization library

components.

2 RELATED WORK

Generally, preserving structure means that a visualization is expected to preserve the field properties and connectivity of the corresponding dataset:

field¹ is a set of values of the same type, e.g. date, location, stations, and precipitation in a weather station dataset.

connectivity is how elements are arranged in a dataset[64], e.g. as 0D (•) points, along a 1D (–) line, in a 2D (■) surface, as a 3D trajectory, a volumetric cube, or as a network.

This paper builds on how structure has traditionally been discussed in visualization and mathematics and encapsulated in visualization library design to propose a uniform interface for encoding structure that supports a broader variety of fields and more rigorously defines how connectivity is preserved. Bertin[7] codified the idea that properties of data and graphics should match, using the Steven’s measurement scales [53] as the case study. Mackinlay and Wilkinson translate algebras on data to grammars for specifying graphics to develop frameworks for designing structure preserving visualization tools [37, 36, 64], while our framework uses the same abstractions for data and graphics. Bartonicek built a tool based on the categorical and algebraic properties of stacked, dodged, and layered bar charts and the underlying data [3]; our work provides a more general framework for describing those properties and propose that sheaves are a better abstraction for reasoning about connectivity constraints ?? than monoids and groups. Similarly to Kindlemann and Scheidegger’s algebraic visualization design (AVD) framework [31], our framework presents a uniform abstract data representation layer in subsection 3.2 for ensuring that the visualization should not change if the data representation (i.e. the data container) changes, defines the conditions under which data is mapped unambiguously to visual encodings [68] in subsection 3.3, and provides a methodology for verifying that changes in data should correspond to changes in the visualization in subsubsection 4.2.2 that does not necessarily require that the changes be perceptually significant. Our model generalizes previous mathematically derived frameworks by allowing arbitrary structures, explicitly incorporating topology, and by providing an explicit framework for translating the theoretical ideas into buildable components in section 5.

2.1 Structure Preservation In Software

Visualization libraries are in part measured by how expressive the components of the library are, where expressiveness is a measure of which structure preserving mappings a tool can implement [37]. The visual algorithms implemented in these libraries are generally assumed to preserve the underlying continuity of their input data, as described in taxonomies of visualization algorithms by Chi[15] and by Troy and Möller [57]. While some visualization tools, such as Tableau[54, 19, 38], aim to automate the pairing of data with structure preserving visual representations, many visualization libraries leave that choice to the user. In building block libraries² such as

¹In this paper, color in definitions, equations, and visualizations is used to group conceptually related terms[24], for example **field** and **fiber**.

²a term introduced by Wongsuphasawat [66, 65] to describe libraries that provide modular components for building elements of a visualization

Matplotlib[30] and D3[9] assumptions about connectivity are embedded in the interfaces such that the API is inconsistent across plot types. For example in Matplotlib methods for updating data and parameters for controlling aesthetics differ between (1D) line based plotting methods and (0D) marker based methods. While libraries like VTK[20, 16] and Bokeh[8] provide a way to express the connectivity of the data, and therefore can embed that information in their visual algorithms, their charting APIs are similar to the continuity dependent APIs of other building block libraries.

Domain specific libraries are designed with the assumption of continuities that are common in the domain [25], and therefore can somewhat restrict their API to choices that are appropriate for that domain. For example, a tabular topological structure of discrete rows, as illustrated in ??, is assumed by A Presentation Tool[37, 37] and grammar of graphics[64] and the ggplot[61], vega[47], and altair[59] libraries built on these frameworks. Image libraries such as Napari[49] and ImageJ[48] and the ImagePlot[55] art plugin assume that the input is a 2D continuous image. Networking libraries such as gephi[4] and networkx[18] assume a graph-like structure. By assuming the structure of their data, these domain specific libraries can provide interfaces that are easier to navigate due to more similar specification notation [33] because the visual algorithms can share assumptions about the underlying data.

We propose that the cohesion of domain specific library APIs is obtainable using the uniform data model described in subsection 3.1 while the expressivity of building block libraries can be preserved by defining explicit structure preserving constraints on the library components, as described in section 4. Because category theory constructions map cleanly to objects and functions, using category theory to express the structure and constraints can lead to more consistent software interfaces in visualization software libraries [63, 67]. A brief visualization oriented introduction to category theory is in Vickers et al [60], but they are applying category theory to semantic concerns about visualization design rather than library architecture.

3 UNIFORM ABSTRACTION FOR DATA & GRAPHICS

In this section, we propose a mathematical abstraction of the data input and pre-rendered graphic output. This mathematical abstraction provides a uniform highly generalizable method for describing topology and fields; expresses how to verify that data connectivity is preserved on subset, distributed, and streaming data representations; and formalizes the expectation of a correspondence between data and visual elements. Using these abstractions allows us to embed information about structure in dataset types:

`dataset : connectivity  $\rightarrow$  fields`

which can then be checked by visualization algorithms to ensure that the assumptions of the data match the assumptions of the algorithm. This typing system also extends to pre-rendered graphic output, allowing us to develop the structure preservation framework in section 4 that ensures that output fields are equivalent to the input fields and that the connectivity of the output graphic matches the connectivity of the input.

3.1 Abstract Data Representation

We model data using a mathematical representation of data that can encode topological properties, field types, and data values in a uniform manner using a structure from algebraic topology called a fiber bundle. We extend Butler’s proposal of using bundles as an abstraction for visualization data[13, 14] by incorporating Spivak’s methodology for encoding named data types from his fiber bundle representation of relational databases [51, 52].

A **fiber bundle** $F \hookrightarrow E \xrightarrow{\pi} K$ is a structure with topological spaces E, F, K and bundle projection map $\pi : E \rightarrow K$ [50](see Supplement section 4 for details) which we use as a data model by:

- encoding the connectivity of the data as a **base space** E modeling the data indexing space
- encoding the fields as a **fiber space** F that acts as the data schema (domain)
- expressing the mappings between these two spaces as **section** τ functions that encode datasets as mappings from indexing space to field space

which provides a uniform interface for individually specifying the connectivity K and field F properties and the relationship between these spaces (dataset: connectivity) $\rightarrow$ fields for datasets with arbitrary structures. Fiber bundle spaces can also be partitioned into sub bundles, called local trivializations, with transition maps to glue the local trivializations back together. For example, we can unwrap a sphere into a plane to show that a continuous map exists between a segment of the globe and a corresponding choropleth, thereby addressing Wilkinson’s concern that connectivity cannot be tested in a generalizable way because continuous maps do not always exist, specifically between a plane and a sphere [64]. The subsets of the fiber bundle and the transition maps between these subsets are encoded in an atlas[17] and the notion of an atlas can be incorporated into the data container, as discussed in subsection 3.2.

3.1.1 Connectivity: Base Space K

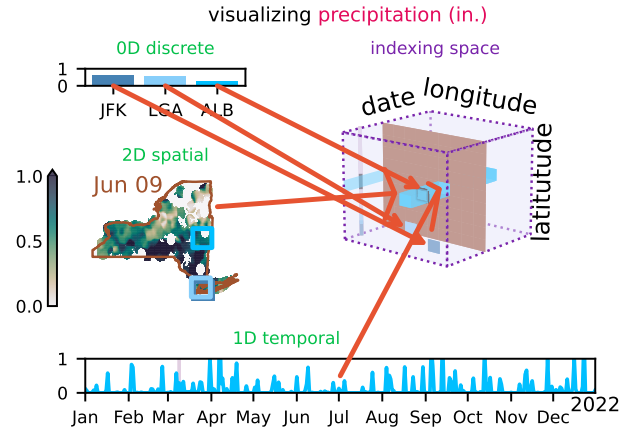


Figure 1: This weather station data has multiple embedded continuities - points at each time and position, timeseries at each position, and maps at each time. The corresponding visualizations - bar chart, timeseries, and map - each preserve the continuity of the subset of the data they visualize by not introducing or leaving out values and preserving the relative positioning of continuous values.

Describing connectivity using the language of topology allows for describing individual elements in a way that holds true whether the data fits in memory, is distributed, or is streaming. This is because, informally, a topology $\mathcal{T}$ on the underlying data indexing space (which is a proxy for the continuity), is a partitioning of that space such that the partitions are of the same mathematical type as each other and the partitioned space. The partitions must also be composable in a continuity and property preserving way. There are various equivalent definitions of topology, but we use the neighborhood axiomatization (Supplement subsection 1.3) because it is most analogous to the data access model of index (element) in subset (neighborhood) of all indices (mathematical space).

Definition 3.1. The **base space** is the indexing space $\mathbf{K}$ such that there is a neighborhood *topology* (partitioning function) $\mathcal{N}$ that assigns elements $k \in \mathbf{K}$ to a collection of opensets $\mathcal{U}$ such that each element of $\mathcal{N}(k)$ is a *neighborhood of k* and $k \subset \mathcal{U} \subset \mathcal{N}(k)$. A neighborhood of an element k has to contain k , can grow, or shrink, and every neighborhood also contains smaller neighborhoods of points adjacent to x .

In the indexing cube in Figure 1, the brown surface and blue rectangle cube are both neighborhoods of the index for the measurement in Albany on June 09. The blue rectangle is also a neighborhood of the index for the measurement in Albany on March 09. The indexing cube is a neighborhood for both of these indices. Using Munzner’s key-value based semantic indexing[41], the data in Figure 1 could be subset into sets of timeseries where the key would be station, or subset into maps where the key would be date, or subset into station records where the keys are (date, latitude, longitude). Using a connectivity model expresses how different labeling schemes can refer to the same indices, for example how 0-360 and 180E-180W are two ways of labeling longitude or how (date, lat, lon) and (date, station) refer to the same data.

While arbitrary topological spaces can have neighborhood topologies, in this paper we usually model the indexing space as CW-complexes. CW-complexes are a class of topological spaces built by gluing together n -dimensional balls (which include points, intervals, filled circles, filled spheres, etc.) using continuous attaching maps. Because the base space of a fiber bundle is a quotient topology[40], it divides the topological space into the largest number of open sets such that π remains a continuous function. This means that the topology can be defined to have a resolution equal to the number of indices in a dataset such that the key (continuity)-value (data) pairing is always preserved.

Base Space Data Type Following from Spivak’s categorical abstraction of a database [52, 51], we also propose that the structure of the data types be formally specified as the objects of a category (Supplement subsection 2.1). The standard construction of a category from a topological space is that it has open set objects $\mathcal{U}$ and morphisms $\hat{\phi} \in \text{Hom}(\mathcal{U}, \mathcal{U})$ [46]. We denote the objects as $\mathbf{K}$ when the underlying fiber bundle is trivial (Supplement subsection 4.1), which is the case for the bundles discussed in this paper. The composability property expresses that the morphism is transitive, while associativity expresses that the morphisms can be carried in various equivalent groupings. By formally specifying the properties of the topological structure data types as $\mathcal{K}$, we can express that these are the properties that are required as part of the implementation of the connectivity data type objects.

3.1.2 Field Data Types: Fiber Space $\mathbf{F}$

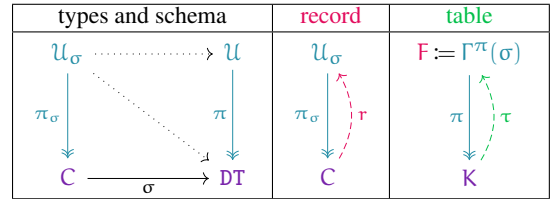
Spivak shows that data typing can be expressed in a categorical framework in his fiber bundle formulation of tables in relational databases [51, 52]. We adopt Spivak’s definitions of *type specification*, *schema*, *record*, and *table* because that allows us to use a dimension agnostic named typing system for the dataset fields that is consistent with the abstraction we are using to express the continuity.

Spivak introduces a *type specification* as a bundle map $\pi : \mathcal{U} \rightarrow \mathbf{DT}$. The base space $\mathbf{DT}$ is a set of data types $T \in \mathbf{DT}$ and the total space $\mathcal{U}$ is the disjoint union of the domains of each type $\mathcal{U} = \bigsqcup_{T \in \mathbf{DT}} \pi^{-1}(T)$ such that each element x in the domain $\pi^{-1}(T)$ is one possible value of an object of type T [52]. For example, if $T = \text{int}$, then the image $\pi^{-1}(\text{int}) = \mathbb{Z} \subset \mathcal{U}$ is the set of all integers and $x = 3 \in \mathbb{Z}$ is the value of one int object. Spivak’s analogue of a database *schema* is the *simple schema of type π* which is defined as consisting of the pair (C, σ) where C is the set of field

names and $\sigma : C \rightarrow \mathbf{DT}$ is a function from field name to field data type[52].

The function σ is composed with π such that $\pi^{-1}(\sigma(C)) \subseteq \mathcal{U}$; this composition induces a domain bundle $\pi_\sigma : \mathcal{U}_\sigma \rightarrow C$ that associates a field name $c \in C$ with its corresponding domain $\pi_\sigma^{-1}(c) \subseteq \mathcal{U}_\sigma$. Spivak defines a *record* as a function $r : C \rightarrow \mathcal{U}_\sigma$ from column name to type domain and the set of records on π_σ is denoted $\Gamma^\pi(\sigma)$ ³. Records must return an object of type $\sigma(C) \in \mathbf{DT}$ for each field $c \in C$. Spivak then describes tables as sections $\tau : K \rightarrow \Gamma^\pi(\sigma)$ from an indexing space K to the set of all possible records $\Gamma^\pi(\sigma)$ on the schema bundle, and his construction of a table generalizes to our notion of a dataset.

In summary, Spivak defines *types* as map σ from type name (e.g. temperature $\in C$) to data type ($\text{float} \in \mathbf{DT}$) and maps π_σ, π from type domain ($\mathbb{R} \subseteq \mathcal{U} \subseteq \mathcal{U}$) to name and type, *schemas* as maps from type name to data type (temperature : float), *records* as maps from name to value (temperature : 152.0°F) and *tables* as maps from index to record [52]:



Building on Spivak’s constructions:

Definition 3.2. the **fiber space $\mathbf{F}$** is the space of all possible data records:

$$\mathbf{F} := \{r : C \rightarrow \mathcal{U}_\sigma \mid \pi_\sigma(r(C)) = C \text{ for all } C \in C\} \quad (1)$$

such that the preimage of a point is the corresponding data type domain $\pi^{-1}(k) = F_k = \mathcal{U}_{\sigma_k}$.

Adopting Spivak’s fiber bundle construction of types allows our model to reuse types so long as the field names are distinct and that field values can be accessed by field name, since those are sections on $\mathcal{U}_\sigma$. Furthermore, since domains $\mathcal{U}_\sigma$ of types are a mathematical space, multi-dimensional fields can be encoded in the same manner as single dimensional fields and fields can have different names but the same type.

Fiber space Data Type As with the base space category $\mathcal{K}$, we propose a fiber category $\mathcal{F}$ to encapsulate the field types of the data. The fiber category has a single object $\mathbf{F}$ of an arbitrary data type (e.g. int) and morphisms on the fiber object $\hat{\phi} \in \text{Hom}(\mathbf{F}, \mathbf{F})$. We can also equip the category with any operators or relations that are part of the mathematical structure of the field type, for example an addition or comparison operator. A single object category is a monoid and therefore also equipped with a bifunctor $\otimes : \text{dfiber} \times \mathbf{F} \rightarrow \mathbf{F}$.

The bifunctor can be defined to express constraints on composite types. For example, dates can be represented as three fields $F_{\text{year}} \times F_{\text{month}} \times F_{\text{day}}$ or a composite fiber field $F_{\text{year}} \times F_{\text{month}} \times F_{\text{day}} = F_{\text{date}}$. The $\otimes$ encapsulates both the sets of values associated with each fiber $\{y \in \mathbb{I} | 1992 \leq y \leq 2025\} \times \{m \in \mathbb{I} | 1 \leq m \leq 12\} \times \{d \in \mathbb{I} | 1 \leq d \leq 31\}$ and the composition function $\otimes : F_{\text{year}} \times F_{\text{month}} \times F_{\text{day}} \rightarrow F_{\text{date}}$ which would include a constraint to only return dates that have the right number of days for each month. The bifunctor also composes the morphisms associated with each category into a morphism on the composite category $(\hat{\phi}_{\text{year}}, \hat{\phi}_{\text{month}}, \hat{\phi}_{\text{day}}) = \hat{\phi}_{\text{date}}$.

³This paper follows the convention that Γ denotes a set.

3.1.3 Composition: Adding and expanding records

A common use of category theory in software engineering is the specification of modular components [63] such that we can build systems where the structure preserved by components is preserved in the composition of the components. As described in [subsubsection 3.1.1](#), the index space is modeled as a set of open sets; therefore adding index spaces K_a and K_b is the coproduct $K_a \sqcup_{K_c} K_b$ over a shared space K_c . As described in [subsubsection 3.1.2](#), the bifunctor can be used to build composite fiber types; in the same manner fields F_a and F_b can be composed by taking the fiber product $F_a \times_{F_c} F_b$ over a shared fiber F_c .

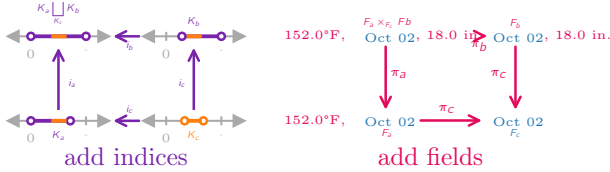


Figure 2: Records are added to the dataset as a *coproduct* $K_a \sqcup_{K_c} K_b$ of index spaces. Fields are added to each record as a *product* $F_a \times_{F_c} F_b$ of fiber spaces.

For example, in [Figure 2](#), the open interval K_c is present in both K_a and K_b ; therefore K_a and K_b are glued together where they overlap at K_c . Two spaces have been combined correctly if every point in the overlap $k \in K_c$ is present in each of the spaces $k \in K_a$ and $k \in K_b$ such that it is present in the joined space $k \in K_a \sqcup_{K_c} K_b$. In [Figure 2](#), the red (temperature, time, pressure) record separates into (temperature, time) and (pressure, time) records that share the same red time. Furthermore this time is the same whether it is obtained from the (temperature, time) or (pressure, time) record. Generally, the test for joining fields is that when a record is present in a fiber $r_c \in \pi_c(F_a), r_c \in \pi_c(F_b)$ then it is in the joint record $r_c \in \pi_c(F_a \times_{F_c} F_b)$ when is in the field F_c being joined on. Verifying that indices are joined correctly allows us to reliably build larger datasets out of smaller ones, such as in the case of distributed and on demand datasets. Verifying that fields are joined together correctly for the same record allows us to reliably combine multiple datasets together on shared properties, for example growing the weather station data in ?? from a temporal to spatial dataset by adding the weather at each location at each time.

3.1.4 Data: Section τ

We encode data as a [section](#) τ of a bundle because this allows us to incorporate the topology and field types in the data definition.

We define section functions locally, meaning that

Definition 3.3. the [section](#) τ is (piece-wise) continuous over a specific open subset U of K

$$\Gamma(U, E|_U) := \{ \tau : U \rightarrow E|_U \mid \pi(\tau(k)) = k \text{ for all } k \in U \} \quad (2)$$

such that each section function $\tau : k \mapsto r$ maps from each point $k \in U$ to a corresponding record in the fiber space $r \in F_k$ over that point.

Bundles can have multiple sections, as denoted by $\Gamma(U, E|_U)$. We can therefore model data as structures that map from an index like point k to a data record r , and encapsulate multiple datasets with the same fiber and base space as different sections of the same bundle.

When a bundle is trivial $E = K \times F$, we can defined a global sections $\tau : K \rightarrow F \in \Gamma(K, F)$ which we translate into a data signature

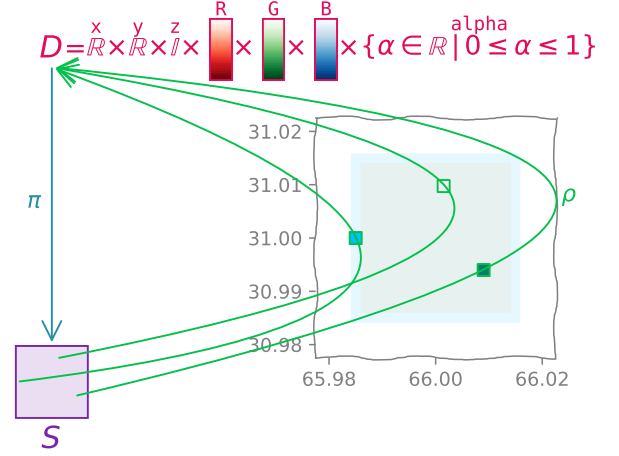


Figure 3: The scatter marker is specified by the section ρ , which maps into the fiber D to retrieve the values that compose into the pixel (approximated as a square) returned by the section function evaluated at each point s . The section evaluated over the entire space $\rho|_S$ returns the entire scatter mark, shown here in faded form to make it easier to see the individual pixels.

of the form [dataset](#) : [topology](#) $\rightarrow$ [field](#) where $\tau = \text{dataset}$, $K = \text{topology}$ and $F = \text{fields}$. When the bundle is non-trivial, we can use the fiber bundle property of local-triviality to define local sections $\tau|_U \in \Gamma(U, E|_U)$. A local section is defined over an open neighborhood $k \in U \in K$, which is an open set that surrounds a point k . Most data sets can be encoded as a collection of local sections $\{ \tau|_{U_k} \mid k \in K \}$ and this encoding can be translated into a set of signatures $\{ \text{data-subset} : \text{topology} \rightarrow \text{fields} \text{ s.t. } \text{data-subset} \subset \text{dataset} \}$.

3.1.5 Example: Uniform Abstract Graphic Representation

One of the reasons we use fiber bundles as an abstraction is that they are general enough that we can also encode the output of a visual algorithm as a bundle. We denote the output as a graphic, but the use of bundles allows us to generalize to output on any display space, such as a screen or 3D print.

$$D \hookrightarrow H \xrightarrow{\pi} S \quad (3)$$

The total space H is an abstraction of an ideal (infinite resolution) space into which the graphic can be rendered. The base space S is a parameterization of the display area, for example the inked bounding box in the cairo [1] rendering library. The fiber space D is an abstraction of the renderer fields; for example a 2 dimension screen has pixels that can be parameterized $D = \{x, y, r, g, b, a\}$. As with data, we model the graphic specifications as sections ρ of the graphic bundle

$$\Gamma(W, H|_W) := \{ \rho : W \rightarrow H|_W \mid \pi(\rho(s)) = s \text{ for all } s \in W \} \quad (4)$$

that map from a point in an openset in the graphic space $s \in W \subseteq S$ to a point in the graphic fiber D . The section evaluated on a single point s returns a single graphic record, for example one pixel in an ideal resolution space. In our model, the unevaluated graphic section is passed from the visualization library component to the renderer to generate graphics.

In [Figure 3](#), the section function ρ maps into the fiber for a simplified 2D RGB infinite resolution pre-render space and returns the

$\{x, y, r, g, b\}$ values of a pixel in an infinite resolution space. In Figure 3 these pixels are approximated as the small colored boxes. Each pixel is the output of the $\rho(s)$ section that intersects the box. The set of all pixels returned by a section evaluated on a given visual base space $\rho|_S$ can yield a visual element, such as a marker, line, or piece of a glyph and in Figure 3 is a blue circle with a black edge. While Figure 3 illustrates a highly idealized space with no overlaps, overlaps can be managed via a fiber element D_z for ordering. It is left to the renderer to choose how to blend D_z and D_a layers.

3.1.6 Data transformations

Data is often codified as the set of values and the mathematical actions (Supplement subsection 1.2) on those values, for example the Steven's measurement scales define nominal, ordinal, interval, and ratio data by the group actions on the data [53]; other researchers have since expanded the scales to encapsulate more types of structure [34, 56]. We separate data transformations into two components, transformations on the base space $(\hat{\phi}, \hat{\phi}^*)$ and transformations on the fiber space $\tilde{\phi}$.

Connectivity The base space transformation is a point wise continuous map from one open set to another open set in the same base space

$$\hat{\phi} : K' \mapsto K \quad (5)$$

such that $U, U' \subseteq K$. This means U and U' are of the same topology type. To correctly align the sections with the remapped base space, there is a corresponding section pullback function

$$\hat{\phi}^* \tau|_{U'} : \tau|_{U'} \mapsto \tau|_{U \circ \hat{\phi}} \quad (6)$$

such that $\tau|_U = \hat{\phi}^* \tau|_{U'}$ because $\tau|_U = \tau|_{\hat{\phi}(U')}$. This means that the base space transformation $\hat{\phi}(K') = \hat{\phi}(K)$ such that

$$\tau(K) = \hat{\phi}^* \tau(K') = \tau(\hat{\phi}(K')) \quad (7)$$

which means that the index of the record changes from k to k' but the values in the record are unmodified.

Records The fiber transformation $\tilde{\phi}$ is a change in section

$$\tilde{\phi} : \hat{\phi}^* \tau|_{U'} \mapsto \hat{\phi}^* \tau'|_{U} \quad (8)$$

where $\tau, \tau' \in \Gamma(U', \hat{\phi}^* E|_{U'})$. Since $\tilde{\phi}$ maps from one continuous function to another, it must itself be continuous such that

$$\lim_{x \rightarrow k'} \tilde{\phi}(\hat{\phi}^* \tau(x)) = \tilde{\phi}(\hat{\phi}^* \tau(k')) \quad (9)$$

As mentioned in subsubsection 3.1.2, $\tilde{\phi}$ is also a morphism on the fiber category $\tilde{\phi} \in \text{Hom}(\hat{\phi}^* F|_{K'}, \hat{\phi}^* F|_{K'})$ restricted to a point $k' \in U'$. This means $\tilde{\phi}$ has to satisfy the properties of a morphism (??)

- *closed*: $\tilde{\phi}(\hat{\phi}^* \tau(k')) \in F$
- *unitality*: $\tilde{\phi}(\text{id}_F(\hat{\phi}^* \tau(k'))) = \text{id}_F(\tilde{\phi}(\hat{\phi}^* \tau(k')))$
- *composition and associativity*:
 $\tilde{\phi}(\tilde{\phi}(\hat{\phi}^* \tau(k'))) = (\tilde{\phi} \circ \tilde{\phi})(\hat{\phi}^* \tau(k'))$

Additionally, $\tilde{\phi}$ must preserve any features of F , such as operators that are defined as part of the structure of F . We do not provide a general rule here because these constraints are defined with respect to how specific properties of the mathematical structure of individual fields F are expected to be preserved rather than as a general consequence of $\tilde{\phi}$ being a section map and morphism of the category.

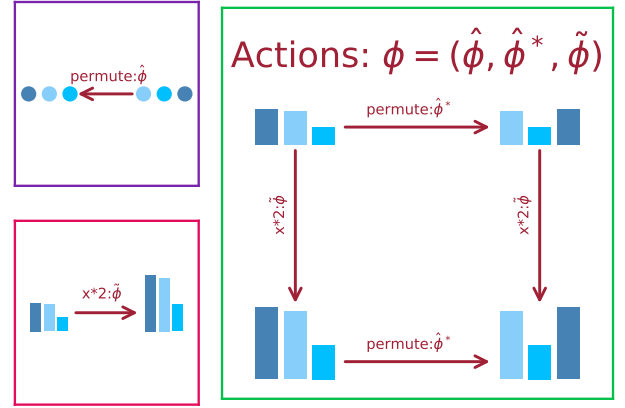


Figure 4: The actions on the base and fiber space commute such that the data can be shuffled then scaled or scaled then shuffled.

Connectivity and Records We define a full data transformation as one that induces both a remapping of the index space and a change in the data values

$$\phi : \tau|_U \mapsto \tau'|_{U \circ \hat{\phi}} \quad (10)$$

which gives us an equation that can express transformations that have both a base space change and a fiber change.

The data transform ϕ is composable

$$\phi = (\hat{\phi}, \prod_{i=0}^n \tilde{\phi}_i) \quad (11)$$

if each (identical) component base space is transformed in the same way $\hat{\phi}$ and there exists functions $\phi_{a,b} : E_a \times E_b \rightarrow E_a \times E_b$, $\phi_a : E_a \rightarrow E_a$ and $\phi_b : E_b \rightarrow E_b$ such that $\pi_a \circ \phi_a = \phi_{a,b} \circ \pi_a$ and $\pi_b \circ \phi_b = \phi_{a,b} \circ \pi_b$ then $\phi_{a,b} = (\phi_a, \phi_b)$. This allows us to define a data transform where each fiber transform $\tilde{\phi}_i$ can be applied to a different fiber field F_i .

The component actions are independent such that they can be applied in either order. In Figure 4, the action $\hat{\phi}, \hat{\phi}^*$ permutes the indexing space such that location of the bars relative to each other changes. The action $\tilde{\phi}$ doubles the data value associated with the height of the bar. Because these are both structure preserving actions, the order in which the bars are transformed - permute then scale or scale then permute - does not change the final image.

3.2 Abstract Data Containers

While bundles provide a way to describe the structure of the data, sheaves are a mathematical way of describing the data container. Sheaves are an algebraic data structure that provides a way of abstractly discussing the bookkeeping that data containers must implement to keep track of the continuity of the data [17]. This is because sheaves and presheaves are functors (Supplement ??), which are mappings between categories that preserve the domains, codomains, composition, and identities of the morphisms within the category [46]. Using sheaves as the data container abstraction facilitates representational invariance, as formulated by Kindlemann and Scheidegger [31], since the container level is uniformly specified as satisfying presheaf and sheaf constraints (Supplement section 5). When a data container satisfies these constraints, the subsets and whole data space have the same mathematical properties, e.g. morphisms, such that the framework in this paper applies whether the data is in memory, distributed, streaming, or on-demand.

Modeling the data container as a functor allows us state that, just like a functor, the container is a map between the index space objects and sets of data records that preserve morphisms between index space objects and data records.

$$\mathcal{O}_{K,E} : \mathcal{U} \rightarrow \Gamma(\mathcal{U}, E|_{\mathcal{U}}) \quad (12)$$

A presheaf is a functor from an arbitrary category to the set category, here used to codify the mapping from index to records. A property of presheaves is that it is contravariant, meaning that the morphisms between the input objects $\hat{\phi}$ go in the opposite direction from the morphisms between the output objects $\hat{\phi}^*$. As discussed in subsection 3.1.6, $\hat{\phi}$ changes the index and $\hat{\phi}^*$ aligns the records by "pulling back" records to where they belong in the new index. The map between sets is contravariant to the map over indices because not all sets exist over all indices, for example when $\hat{\phi}$ is an inclusion mapping that returns a subset of the full index. Sheaves are presheaves that satisfy the gluing and locality axioms. The gluing axiom says that a distributed representation of a dataset, which is a set of local sections, is equivalent to a section over the union of the opensets of the local sections. The gluing axiom can also be used to generate the gluing rules used to construct non-trivial bundles from the set of trivial local sections. The locality axiom asserts that the glued section function is equivalent to a function over the union if they evaluate to the same values. When satisfied, these axioms ensure that a data container is managing distributed and streaming data in a structure preserving way.

The advantage of using these constraints for specifying structure preserving data contains is that the test for whether a container implements these constraints is trivial:

axiom	applied to datasets and indexes
presheaf	given $\text{index}_1 \subset \text{index}_2$: $\exists \text{dataset}[\text{index}_2]$ $\Rightarrow \text{dataset}[\text{index}_2][\text{index}_1] = \text{dataset}[\text{index}_1]$ $\exists \text{dataset}[\text{index}_1] \Rightarrow \exists \text{dataset}[\text{index}_2]$
locality	$\text{dataset}^1[i] = \text{dataset}^2[i] \forall i \in \text{index}$ $\Rightarrow \text{dataset}^1 = \text{dataset}^2$
gluing	$i = j$ and $\text{dataset}^1[i] = \text{dataset}^2[j]$ $\text{dataset}^3 := \text{dataset}^1[i] \oplus \text{dataset}^2[j]$

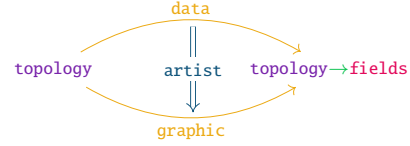
Table 1: Example of testing whether a data container supports inclusion/subsetting in a manner that satisfies sheaf constraints.

While each section of a sheaf is evaluated over a point $\tau(k)$ such that it returns a single record, the sheaf over a very small region surrounding a point k is called a *stalk* and the *germ* is the section evaluated at a point in the stalk. Since the stalk includes the values near the limit of the point at k the germ can be used to compute the mathematical derivative of the data for visualization tasks that require this information.

4 CODIFYING STRUCTURE PRESERVATION

Data visualization components are implementing a structure preserving mapping from a dataset to a graphic, which in the framework presented in this paper is a function from data to graphic sheaf. We call these structure preserving functions **Artists**⁴:

⁴ Artists are named for the Artist class in Matplotlib. Artist is the parent class of all the objects that assemble and manage visual elements such as points, lines, and shapes.



$$A : \Gamma(K, E) \rightarrow \text{Im}_A(S, H), \text{Im}_A(S, H) \subset \Gamma(S, H) \quad (13)$$

The output of an artist A is a restricted subset of graphic sections $\text{Im}_A(S, H)$ that are, by definition, only reachable through a structure preserving artist:

$$\text{Im}_A(S, H) := \{\rho \mid \exists \tau \in \Gamma(K, E) \text{ s.t. } A(\tau) = \rho, \xi(S) = K\} \quad (14)$$

We define this subset because the space of all sections $\Gamma(W, H|_{\mathcal{U}})$ is every possible way of setting pixels on an H base space, including non structure preserving settings such as setting all the pixels the same color.

While Artists are defined as maps going from data over K to graphic over S they are compatible with maps defined solely over K or S . This is because when a sheaf is equipped with transport functors (ξ_*, ξ^*) , maps between sheaves over one space $\text{Hom}_{\mathcal{O}_K}$ are isomorphic to maps between sheaves over the other space $\text{Hom}_{\mathcal{O}_S}$ [21]:

$$\begin{array}{ccc} \Gamma(\mathcal{U}, \xi_* H|_{\mathcal{U}}) & \xleftarrow{\xi_*} & \Gamma(W, H|_{\mathcal{W}}) \\ \text{Hom}_{\mathcal{O}_K} \uparrow & \text{Hom}_{\mathcal{O}_K, \mathcal{O}_S} & \uparrow \text{Hom}_{\mathcal{O}_S} \\ \Gamma(\mathcal{U}, E|_{\mathcal{U}}) & \xrightarrow{\xi^*} & \Gamma(W, \xi^* E|_{\mathcal{W}}) \end{array} \quad (15)$$

Both $\text{Hom}_{\mathcal{O}_K}$ and $\text{Hom}_{\mathcal{O}_S}$ are compatible, via composition with ξ_* , with a mapping from data sheaf in data space to graphic sheaf in graphic space $\text{Hom}_{\mathcal{O}_K, \mathcal{O}_S}$. Artists are a subset of structure preserving functions in $\text{Hom}_{\mathcal{O}_K, \mathcal{O}_S}$. For example, the artist in Figure 5 is the blue arrow that converts $(i, \sin(i), \cos(i))$ into the corresponding sections of the circle and lines.

The commutativity of Equation 22 means that an artist over data space $A_K : \tau \mapsto \xi_* \rho$, an artist over graphic space $A_S : \xi^* \tau \mapsto \rho$, and an artist $A : \tau \mapsto \rho$ are equivalent. Specifically, given Equation 19 (ξ^*) and Equation 17 (ξ_*), $A_K(\tau(k)) = A_S(\xi^* \tau(s)) = A(\tau(k))$ when $\xi(s) = k$. The ability to shift base spaces using adaptors (ξ_*, ξ^*) allows a developer to specify structure preserving visualization components in data space, graphic space, or a mix of both depending on what is optimal for the specific use case. For example, mapping a line to a color in data space, but drawing the line in graphic space so that it can be dynamically resampled based on panning and zooming.

Because Artists can be constructed as morphisms of sheaves over either K or S through the application of pushforward and pullback functors (ξ_*, ξ^*) , they are natural transformations [11].

Definition 4.1. Given two functors $F, G : \mathcal{C} \rightarrow \mathcal{D}$, a **natural transformation** $\alpha : F \rightarrow G$ is a function which assigns to each object c of $\mathcal{C}$ a morphism $\alpha_c : F(\text{input}) \rightarrow G(c)$, $G(c) \in \mathcal{D}$, in such a way that for every morphism $f : c \rightarrow c', c' \in \mathcal{C}$, the morphisms in $\mathcal{D}$ commute such that $\alpha_{c'}(F(f)(F(c))) = G(f)(\alpha_c(F(c)))$. When this holds, α_c is *natural* in c . [35].

This means that the structure on $\mathcal{C}$, as defined by its input objects $c \in \mathcal{C}$ and morphisms $f : c \rightarrow c'$ are preserved in the output of the functors F, G and by maps between functors α . The artist A is the natural transform mapping the sheaf functors $\mathcal{O}_{\text{data}} \rightarrow \mathcal{O}_{\text{graphic}}$,

which both have the signature $K^{\text{op}}(S^{\text{op}}) \rightarrow \text{set } ??$. This preservation of the structure of the input in the output encapsulates the structure preservation discussed in section 2 and serves as the basis for describing structure preservation in subsection 4.2. Because the Artist is a natural transform and natural transforms are maps of functors that take the same input object and return objects in the same category[39], the artist signature can be formulated as:

$$\text{Artist} : (\text{dataset} : \text{topology} \rightarrow \text{fields}) \\ \rightarrow (\text{graphic} : \text{topology} \rightarrow \text{fields})$$

where `topology` and `field` are meta classes of compatible types.

4.1 Homeomorphism

As introduced in ??, the topology of a dataset is preserved in the visualization if there is a homeomorphic map between the graphic and data indexing space. Because multiple graphics can correspond to one data record and every graphic must have a corresponding record [58, 68], we define **indexing look up functor** ξ as a deformation retraction[23] of the graphic indexing space S into the data indexing space K . This means that every point in S can be mapped, non-uniquely, to a point in K

$$\xi : K \times I \rightarrow K \text{ s.t. } \xi(s) = k \forall k \in S \quad (16)$$

The graphic indexing space is defined as the data space multiplied by an identity $S = K \times I$ because it allows us to encode the data index into the graphic index. For example, for a 2D space the graphic basepoint is defined as $s = (\alpha, \beta) \in k_i \times I$. This means that the first coordinate $\alpha = k_i$ provides a lookup into the data space and the second coordinate is a point on the interval $\beta \in I$ and together they are input into the graphic function $\rho(s)$ which returns how the pixel at s is set.

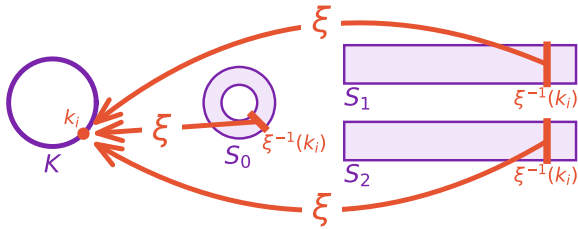


Figure 5: The function ξ maps each point in each orange band on each graphic indexing space S_0, S_1, S_2 to the same point k_i in the circle K . As shown in Figure 5, the parts of the graphic indexed by the orange bands are visualizing parts of the record at k_i

In Figure 6 each orange band is defined as $\frac{7\pi}{4} \times [0, 1]$ such that each point on each band maps to the same point $\xi|_{\frac{7\pi}{4} \times [0, 1]} = \frac{7\pi}{2}$. As illustrated in Figure 5, each band acts as an indexer into its corresponding image because the section ρ takes as input coordinates $(\frac{7\pi}{4}, \beta), \beta \in [0, 1]$ in each orange band and returns the settings for the pixel it maps into. As illustrated in Figure 6, the definition of ξ allows for some flexibility in the shape of S . An annulus S_0 is used for the green circle in Figure 5 because that cyclical structure is in the graphic and in K but rectangles S_1, S_2 are used for the sin and cos curves because they are linear. Despite discarding the cyclical information embedded in K the spaces S_1 and S_2 are structure preserving because they satisfy the constraint on ξ that every graphic element maps back into a data element [68].

Because the artist is specified as a natural transform (Definition 4.1) on sheaves, the artist expects that subsets of the dataset

have the same properties as the whole dataset when the artist preserves the presheaf and sheaf constraints (??, ??) when mapping data into graphics. Preserving the presheaf constraints means that the set of possible visualizations for a larger set of values must include all visualizations for a subset of those values $A(\Gamma(K_1, E)) \xrightarrow{i^*} A(\Gamma(K_2, E))$ when more values may exist on the subset of the index $K_2 \xrightarrow{i} K_1$. While the curves in ?? are meant as stand-ins for the data, they also serve to illustrate that the relationship is identical in the graphic space. The artist preserves the *locality* axiom in that identical datasets $\tau^a = \tau^b$ generate identical specifications $A(\tau^a) = A(\tau^b)$. The artist respects *gluing* in that graphics are composable, as described in subsection 4.3, such that a set of artists that generate graphics for parts of a dataset are equivalent to an artist that generates a graphic for the whole dataset. Managing subsets of data and on demand data in a structure preserving way is necessary to support visualizing streaming data [32] and techniques that rely on shifting or resampling the indexing space, such as panning and zooming [42].

4.2 Equivariance

As introduced in section 2, data and the corresponding visual encoding are expected to have compatible structure such that actions (??) on data and graphic are symmetric. We generalize from binary operations to a family of actions $\phi \in \Phi$ on the sheaf $\mathcal{O}_{K, E}$ because that allows for expanding the set of allowable transformations on the data beyond a single operator. We describe the changes on the graphic side as changes in **measurements** M of visual components. A superset of the Bertin visual variables [6], measurable components are scalar or vector components of the rendered graphic that can be quantified, such as the color, position, shape, texture, or rotation angle of the graphic. For example, a measurement of a scatter marker could be its color (e.g. red) or its x position (e.g. 5).

4.2.1 Mathematical Structure of Data

something something rotation etc We separate data transformations into two components, transformations on the base space $(\hat{\phi}, \hat{\phi}^*)$ and transformations on the fiber space $\tilde{\phi}$.

$$\begin{array}{ccc} \Gamma(U, E|_U) & \xrightarrow{\hat{\phi}^*} & \Gamma(U', \hat{\phi}^* E|_{U'}) \\ \uparrow U & & \uparrow U' \\ U & \xleftarrow{\hat{\phi}} & U' \\ & & \downarrow \tilde{\phi} \\ & & \Gamma(U', \hat{\phi}^* E|_{U'}) \end{array} \quad (17)$$

The base space transformation transforms one openset object U' to another object U , and the pullback functor transports the entire set of sections $\Gamma(U, E|_U)$ over the new base space $\Gamma(U', \hat{\phi}^* E|_{U'})$. The fiber transformation transforms a single section $\hat{\phi}^* \tau$ to a different section $\hat{\phi}^* \tau$.

Records As introduced in Equation 24, the fiber transformation $\tilde{\phi}$ is a change in section

$$\tilde{\phi} : \hat{\phi}^* \tau|_{U'} \mapsto \hat{\phi}^* \tau'|_{U'} \quad (18)$$

where $\tau, \tau' \in \Gamma(U', \hat{\phi}^* E|_{U'})$. Since $\tilde{\phi}$ maps from one continuous function to another, it must itself be continuous such that

$$\lim_{x \rightarrow k'} \tilde{\phi}(\hat{\phi}^* \tau(x)) = \tilde{\phi}(\hat{\phi}^* \tau(k')) \quad (19)$$

As mentioned in subsubsection 3.1.2, $\tilde{\phi}$ is also a morphism on the fiber category $\tilde{\phi} \in \text{Hom}(\hat{\phi}^* F|_{k'}, \hat{\phi}^* F|_{k'})$ restricted to a point $k' \in U'$. This means $\tilde{\phi}$ has to satisfy the properties of a morphism (??)

- *closed*: $\tilde{\phi}(\hat{\phi}^*\tau(k')) \in F$
- *unitality*: $\tilde{\phi}(\text{id}_F(\hat{\phi}^*\tau(k'))) = \text{id}_F(\tilde{\phi}(\hat{\phi}^*\tau(k')))$
- *composition and associativity*:
 $\tilde{\phi}(\tilde{\phi}(\hat{\phi}^*\tau(k'))) = (\tilde{\phi} \circ \tilde{\phi})(\hat{\phi}^*\tau(k'))$

Additionally, $\tilde{\phi}$ must preserve any features of F , such as operators that are defined as part of the structure of F . Examples of testing that $\tilde{\phi}$ preserves the operations, and therefore structure, of the Steven's measurement scales are shown in ???. We do not provide a general rule here because these constraints are defined with respect to how specific properties of the mathematical structure of individual fields F are expected to be preserved rather than as a general consequence of $\tilde{\phi}$ being a section map and morphism of the category.

Topological structure and records We define a full data transformation as one that induces both a remapping of the index space and a change in the data values

$$\phi : \tau \downarrow_U \mapsto \tau' \downarrow_U \circ \hat{\phi} \quad (20)$$

which gives us an equation that can express transformations that have both a base space change and a fiber change.

The data transform ϕ is composable

$$\phi = (\hat{\phi}, \prod_{i=0}^n \tilde{\phi}_i) \quad (21)$$

if each (identical) component base space is transformed in the same way $\hat{\phi}$ and there exists functions $\phi_{a,b} : E_a \times E_b \rightarrow E_a \times E_b$, $\phi_a : E_a \rightarrow E_a$ and $\phi_b : E_b \rightarrow E_b$ such that $\pi_a \circ \phi_a = \phi_{a,b} \circ \pi_a$ and $\pi_b \circ \phi_b = \phi_{a,b} \circ \pi_b$ then $\phi_{a,b} = (\phi_a, \phi_b)$. This allows us to define a data transform where each fiber transform $\tilde{\phi}_i$ can be applied to a different fiber field F_i .

$\tau = \text{data}$			$\hat{\phi}_E^* \tau = \text{data.T}$		$\tilde{\phi}_E \tau = \text{data} * 2$			$\phi_E \tau = \text{data.T} * 2$	
0	1	2	0	3	0	2	4	0	6
			1	4				2	8
3	4	5	2	5	6	8	10	4	10

Figure 6: Values in a data set can be transformed in three ways: $\hat{\phi}$ -values can change position, e.g. transposed; $\tilde{\phi}$ -values can change, e.g. doubled; ϕ -values can change position and value

Figure 7 provides an example of a transposition base space change $\hat{\phi}$, a scaling fiber space change $\tilde{\phi}$, and a composition of the two ϕ applied to each data point $x_k \in \text{data}$. In the transposition only case, the values in $\hat{\phi}^* \tau$ retain their neighbors from τ because ϕ does not change the continuity. Each value in $\hat{\phi}^* \tau$ is also the same as in τ , just moved to the new position. In $\tilde{\phi} \tau$, each value is scaled by two but remains in the same location as in τ . And in $\phi \tau$ each function is transposed such that it retains its neighbors and all values are scaled consistently.

4.2.2 Equivariant Artist

We formalize this structure preservation as equivariance, which is that for every morphism on the data $(\hat{\phi}_E, \tilde{\phi}_E)$ there is an equivalent morphism on the graphic $(\hat{\phi}_H, \tilde{\phi}_H)$. The artist is an equivariant map if the diagram commutes for all points $s' \in S'$

$$\begin{array}{ccc}
 \Gamma(K, E) & \xrightarrow{\Lambda} & \text{Im}_A(S, H) \\
 \hat{\phi}_E^* \downarrow & & \downarrow \hat{\phi}_H^* \\
 \Gamma(K', \hat{\phi}_E^* E) & \xleftarrow{\xi} & \text{Im}_A(S', \hat{\phi}_H^* H) \\
 \hat{\phi}_E \uparrow & & \uparrow \hat{\phi}_H \\
 K' & \xleftarrow{\xi} & S' \\
 \tilde{\phi}_E \downarrow & & \downarrow \tilde{\phi}_H \\
 \Gamma(K', E') & \xrightarrow{\Lambda} & \text{Im}_A(S', H')
 \end{array} \quad (22)$$

such that starting at an arbitrary data point $\tau(k)$ and transforming it into a different data point and then into a graphic

$$A(\tilde{\phi}_E(\tau(\hat{\phi}_E(\xi(s'))))) = \tilde{\phi}_H(A(\tau(\xi(\hat{\phi}_H(s')))))$$

is equivalent to transforming the original data point into a graphic and then transforming the graphic into another graphic. The function $\hat{\phi}_H$ induces a change in graphic generating function that matches the change in data. The graphic transformation $\hat{\phi}_H$ is difficult to define because by definition it acts on a single record, for example a pixel in an idealized 2D screen.

Instead, we define an output **verification** function δ that takes as input the section evaluated on all the graphic space associated with a point $\rho_{\xi^{-1} \downarrow_k}$ and returns the corresponding **measurable visual components** M_k . **formal define M as a space of measurements**

$$\delta : (\rho \circ \xi^{-1}) \mapsto (K \xrightarrow{\delta_\rho} M) \quad (23)$$

The measurable elements can only be computed over the entire preimage because these aspects, such as thickness or marker shape, refer to the entire visual element.

$$\begin{array}{ccc}
 \Gamma(K, E) & \xrightarrow{\Lambda} & \text{Im}_A(S, H) \\
 \eta \downarrow & \delta \swarrow & \downarrow \text{render} \\
 \text{Hom}(K, M) & \xleftarrow{\text{measure}} & \text{visualization}
 \end{array} \quad (24)$$

The extraction function is equivalent to measuring components of the rendered image $\delta = \text{measure} \circ \text{render}$, which means an alternative way of implementing the function when S is not accessible is by decomposing the output into its measurable components.

We also introduce a function η that maps data to the measurement space directly

$$\eta : \tau \mapsto (K \xrightarrow{\eta_\tau} M) \quad (25)$$

such that $\eta_\tau(k)$ is the expected set of measurements M_k . The pair of **verification functions** (η, δ) can be used to test that the expected encoding η_τ of the data matches the actual encoding δ_ρ

$$\eta(\tau)(k) = \delta(A(\tau))(k) = \delta(\rho \circ \xi^{-1})(k) = M_k \quad (26)$$

An artist is equivariant when changes to the input and output are equivariant. As introduced in Equation 5, the base space transformation $\hat{\phi}$ is invariant because $\tau \downarrow_U = \tau \downarrow_{\hat{\phi}(U')}$. This means that, for all points in the data $k \in K$, the measurement should not change if only the base space is transformed

$$\eta(\tau)(\hat{\phi}(k')) = \delta(A(\tau))(k) \quad (27)$$

On the other hand, a change in sections Equation 25 induces an equivalent change in measurements

$$\eta(\tilde{\phi}(\tau))(k) = \tilde{\phi}_M(\delta(A(\tau))(k)) \quad (28)$$

The change in measurements $\tilde{\phi}_M$ is defined by the developer as the symmetry between data and graphic that the artist is expected to preserve.

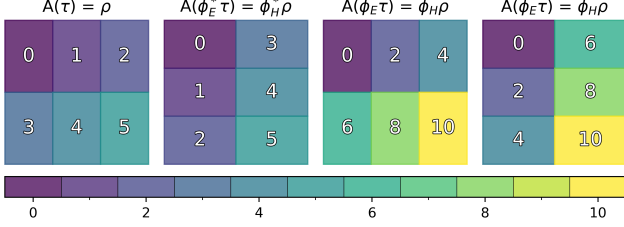


Figure 7: This artist is equivariant because when the input data τ is transposed, $\hat{\phi}$, scaled $\hat{\phi}$, and transposed and scaled ϕ , the corresponding colored cells are transposed, scaled such that the color is moved two steps, and both transposed and scaled.

For example, in Figure 8, the measurable variable is color. This is a visual representation of the data shown in Figure 7, and as such the equivariant transformations are an equivalent transposition and scaling of the colors. This visualization is equivariant with respect to base space transformations, as defined in Equation 34, because the color values at the new position at the old position $\text{measure}'_k = M_k$. This visualization is also equivariant with respect to fiber wise transformations, as defined in Equation 35, because the colors are consistently scaled in the same as the data. For example, the values that have become 2 and 4 in the $\hat{\phi}$ and ϕ panels are colored the same as the original 2 and 4 values in the first panel. The equivariance in this visualization is composable, as shown in the colors being both transposed and scaled correctly in the ϕ panel.

4.3 Composing Artists

addition: intersections mapped same, multiplication: fibers mapped same large big data glued together correctly This allows us to express that an artist that works on a dataset can be composed of artists that work on sub parts of that dataset.

4.3.1 Addition

We propose an addition operator that states that an artist that takes in a dataset can be constructed using artists that take as inputs subsets of the dataset **adjoint equation is wrong, probably multiplication b/c using K^a and K^b to define the piecewise, disjoint union is coproduct**

$$A_{a+b}(\Gamma(K^a \sqcup_{K^c} K^b, E)) := A_a(\Gamma(K^a, E)) + A_b(\Gamma(K^b, E))$$

As introduce in Equation 20, the artist returns a function ρ . We assume that the output space is a trivial bundle, which means that $\rho \in \text{Hom}(S, D)$ because the output specification is the same at each point S . This allows us to make use of the hom set adjoint property **find citation**

$$\text{Hom}(S^a + S^b, D) = \text{Hom}(S^a, D) + \text{Hom}(S^b, D)$$

to define an artist constructed via addition as consisting of two distinct graphic sections

$$\rho(s) := \begin{cases} \rho^a(s) & s \in \xi^{-1}(K^a) \\ \rho^b(s) & s \in \xi^{-1}(K^b) \end{cases} \quad (29)$$

that are evaluated only if the input graphic point is an the graphic area that graphic section acts on.

One way to verify that these artists are composable is to check that the return the same graphic on points in the intersection K^c . Given $k_a \in K_c \subset K_a$ and $k_b \in K_c \subset K_b$, if $k_a = k_b$ then

$$\begin{aligned} A_{a+b}(\tau^{a+b}(k_a)) \\ = A_a(\tau^a(k_a)) = A_b(\tau^b(k_b)) \end{aligned} \quad (30)$$

for all $k_a, k_b \in K_a \sqcup_{K^c} K_b$

replace w/ a line plot w/markers One example of an artist that is a sum of artists is a sphere drawer that draws different quadrants of a sphere $A(\tau) = A_1(\tau_1) + A_2(\tau_2) + A_3(\tau_3)A_4(\tau_4)$. Given an input $k \in K_4$ in the 4th quadrant, then the graphic section that would be executed is ρ_4 . If that point is also in the 3rd quadrant $k \in K_3$, then both artist outputs must return the same values $\rho_4(\xi^{-1}(k)) = \rho_3(\xi^{-1}(k))$.

4.3.2 Multiplication

fiber product vs cartesian product

In the trivial case where the base spaces are the same $K^a = K^b = K$, this is equivalent to adding more fields to a dataset.

$$A_{a \times b}(\Gamma(K, E^{a \times b})) := A_a(\Gamma(K, E^a)) \times A_b(\Gamma(K, E^b))$$

which following from an adjoint property of homsets **find citation and push this into a footnote or appendix maybe**

$$\text{Hom}(S, D) \times \text{Hom}(S, D) = \text{Hom}(S, D \times D) \quad (31)$$

which means that the artists on the subsets of fibers can be defined

$$\rho^{a \times b} = \{\rho^a(s), \rho^b(s)\}, s \in \xi^{-1}(K) \quad (32)$$

but that the signature of $\rho^{a \times b}$ would be $S \rightarrow D \times D$. Instead of having to special case the return type of artists that are compositions of multiple case, the hom adjoint **find cite** property

$$\text{Hom}(S, D \times D) = \text{Hom}(S + S, D)$$

means that multiplication can be considered as a special case of addition where $K^a = K^b$. While we discussed the trivial case in subsection 4.3.1, there is no strict requirement that $F^a = F^b$.

One way to verify that these artists are composable is to check that they encode any shared fiber F^c in the same way.

$$\begin{aligned} \delta(A_{a \times b}(\tau^{a \times b}(k))) \upharpoonright_{F^c} \\ = \delta(A_a(\tau^a(k_a))) \upharpoonright_{F^c} = \delta(A_b(\tau^b(k_b))) \upharpoonright_{F^c} \end{aligned} \quad (33)$$

This expectation of using the same encoding for the same variable is a generalization of the concept of consistency checking of multiple view encodings discussed by Qu and Hullman [43]. This expectation can also be used to check that a multipart glyph is assembled correctly. For example, a box plot [62] typically consists of a rectangle, multiple lines, and scatter points; therefore a boxplot artist $A_{\text{boxplot}} = A_{\text{rect}} \times A_{\text{errors}} \times A_{\text{line}} \times A_{\text{points}}$ must be constructed such that all the sub artists draw a graphic at or around the same x value.

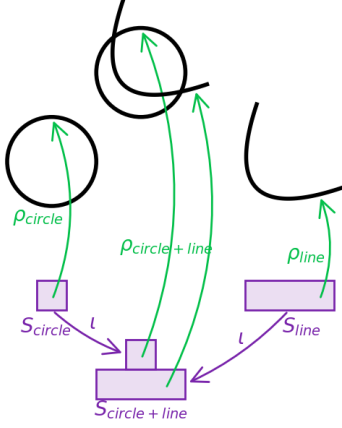


Figure 8: The circle-line visual element can be constructed via $\rho_{\text{circle}} + \rho_{\text{line}}$ functions that generate the circle and line elements respectively. This is equivalent to a $\rho_{\text{circle+line}}$ function that takes as input the combined base space $S_{\text{circle}} \sqcup S_{\text{line}} = S_{\text{circle+line}}$ and returns pixels in the circle-line element.

There is no way to visually determine whether a visual element is the output of a single artist or a multiplied or added collection of artists. The circle-line visual element in Figure 9 can be a visual representation of a highlighted point intersecting with a line plot with the same fields. The same element can also be encoding some fields of a section in the circle and other fields of that section in the lines. *+*equiv* Although we have been discussing the trivial cases of adding observations or adding fields, this merging of artists in datasets can be generalized:

$$A(\Gamma(\sqcup_i K^i, \oplus_i E^i)) := \sum_i A_i(\Gamma(K^i, E^i)) \quad (34)$$

As shown in Equation 41, bundles over a union of base spaces can be joined as a product of the fibers. This allows us to consider all the data inputs in a complex visualization as a combined input, where some sections evaluate to null in fields for which there are no values for that point in the combined base space $k \in \sqcup_i K^i$. The combined construction of the data is a method for expressing what each data input has in common with another data input—for example the data for labeling tick marks or legends— and therefore which commonalities need to be preserved in the artists that act on these inputs.

explain why annotation is similar to brush/linking in operators section

4.4 Animation and Interactivity

pan, zoom, scroll sheaf: locality + gluing ??

selection and hover pushforward Equation 17, pullback Equation 19

brushing, linking, annotation composition of artists Equation 41

Animation and interaction are a set of stills. Because the constraints are on the functions $A \circ \tau$, satisfying the constraints on each function means that the constraint is satisfied for all visualizations $\{A(\tau(k)) \mid k \in K\}$ that make up an animation or interaction.

5 CONSTRUCTING STRUCTURE PRESERVING COMPONENTS

add a high level diagram Data- $\hookrightarrow$ V- $\hookrightarrow$ Screen add back in path of Q, use tikz backend to convert to pgf to then tweak We propose that

one way of constructing artist functions is to separate generating a visualization into an encoding stage ν and a compositing stage Q . In the encoding stage ν , a data bundle is treated as separable fields and each field is mapped to a measurable visual variable. In the encoding stage, many of the expected visual mappings η can be implemented inside the library. Factoring out the encoding stage leaves the compositing stage Q responsible for faithfully translating those measurable visual components into a visual element.

As mentioned in ??, we construct the data base space as a deformation retraction of the graphic space. On simple way of doing so is to construct the graphic base space as a constant multiple of the base space such that

$$\underbrace{K \times [0, 1]^n}_S \xrightarrow{\xi} K \quad (35)$$

where n is a thickening of the graphic base space S to account for the dimensionality of the output space

$$n = \begin{cases} \dim(S) - \dim(K) & \dim(K) < \dim(S) \\ 0 & \text{otherwise} \end{cases}$$

because otherwise the data dimensionality K may be too small for a graphic representation. For example, as shown in ??, a line is 1D but is a 2D glyph on a screen; therefore the graphic space S is constructed by multiplying the base space K with an interval $[0, 1]$.

5.1 Measurable Visual Components

We encapsulate the space of measurable components reachable through the encoding stage ν as a visual fiber bundle $P \hookrightarrow V \xrightarrow{\pi} K$. The restricted fiber space P of the bundle acts as the specification of the internal library representation of the measurable visual components. The space of visual sections $\Gamma(U, V \upharpoonright_U) := \{\mu: U \rightarrow V \upharpoonright_U \mid \pi(\mu(k)) = k \text{ for all } k \in U\}$ return a visual encoding $\mu(k)$ corresponding to data record $k(k)$. Since the data bundle dtotal and visual bundle V have the same continuity $\pi(\tau(k)) = \pi(\mu(k))$, they are considered structurally equivalent such that $E = V$. The distinguishing characteristic of V is that it is part of the construction of the artist and therefore a part of the visualization library implementation. We propose that reusing the fibers P across components facilitates standardizing internal types across the library and that this standardization improves maintainability (??).

5.2 Component Encoders

As introduced in subsection 4.2, there is a set η of functions that map between data and corresponding visual encodings. We propose that for visualization library components to be structure preserving, they must implement a constrained subset of these encoding functions

$$\Gamma(K, E) \xrightarrow{\nu} \Gamma(K, V) \subset \Gamma(K, E) \xrightarrow{\eta} \text{Hom}(K, M) \quad (36)$$

that preserve the categorical structure (operators and morphisms) of the fiber and the continuity of the data section. As mentioned in subsection 5.1, the total visual space is restricted to the space of data types internal to the library $P \subset M$ and sections are subsets of homsets $\Gamma(K, V) \subset \text{Hom}(K, M)$ because sections must be continuous.

The encoding functions ν are fiber wise transforms such that $\pi(E) = \pi(\nu(E))$. A consequence of this property is that ν can be constructed as a point wise transformation such that

$$\nu: F_k \rightarrow P_k \quad (37)$$

which means that means that a point in a single data fiber $r \in F_k$ can be mapped into a corresponding point in a visual fiber $V \in P_k$.

This means that an encoding function v can convert a single record independent of the whole dataset.

Since E and v_{total} are structurally identical, any V can be redefined as E ; therefore, as shown in Equation 45, any collection of v functions can be composed such that they are equivalent to a v that directly converts the input to the output.

$$F_k \xrightarrow{v} P_k \coloneqq F'_k \xrightarrow{v'} P'_k \quad (38)$$

As with artists, v are maps of sections such that the operators defined in subsection 4.3 can also act on transformers v , meaning that encoders can be added $v_{a+b} = v_a + v_b$ and multiplied $v_{a \times b} = v_a v_b$. Encoders designed to satisfy these composability constraints provide for a rich set of building blocks for implementing complex encoders.

5.2.1 Encoder Verification

A motivation for constructing an artist with an encoder stage v is so that the conversion from data to measurable component can be tested separately from the assembly of components into a glyph.

$$\begin{array}{ccc} & \eta_{ab} & \\ F_k^a \times F_k^b & \xrightarrow{v_{ab}} & P_k^a \times P_k^b \\ \pi_a \downarrow & & \downarrow \pi_a \\ F_k^a & \xrightarrow{v_a} & P_k^a \\ \pi_a \uparrow & & \uparrow \pi_a \\ F_k^a \times F_k^c & \xrightarrow{v_{ac}} & P_k^a \times P_k^c \\ & \eta_{ac} & \end{array} \quad \begin{array}{ccc} & & M_k^{ab} \\ & & \downarrow M|_a \\ & & M_k^a \\ & \approx & \\ & & \uparrow M|_a \\ & & M_k^{ac} \end{array} \quad (39)$$

As shown in Equation 46, an encoder is considered valid if there is an isomorphism between the actual outputted visual component and the expected measurable component encoding. An encoder is consistent if it encodes the same field in the same way even if coming from different data sources.

An encoding function v is equivariant if the change in data, as defined in subsubsection 4.2.1, and change in visual components are equivariant. Since E and V are over the same base space and are point wise, the base space change $\hat{\phi}_E$ applies to both sides of the equation

$$v(\tau_E(\hat{\phi}_K(k'))) = \mu(\hat{\phi}_K(k')) \quad (40)$$

and therefore there should not be a change in encoding. On the other hand, a change in the data values $\hat{\phi}_E$ must have an equivalent change in visual components

$$\tilde{\phi}_V v(\tau(k)) = v(\tilde{\phi}_E(\tau(k))) \quad (41)$$

The change in visual components $\tilde{\phi}_V$ is dependent both on $\tilde{\phi}_E$ and the choice of visual encoding. As mentioned in section 2, this is why Bertin and many others since have advocated choosing an encoding that has a structure that matches the data structure[7]. For example choosing a quantitative color map to encode quantitative data if the $\tilde{\phi}$ operation is scaling, as in Figure 8.

5.3 Graphic Compositor

The compositor function Q transforms the measurable components into properties of a visual element. The compositing function Q

transforms the sections of visual elements μ into sections of graphics ρ .

$$Q : \Gamma(K, V) \rightarrow \Gamma(S, H) \quad (42)$$

The compositing function is map from sheaves over K to sheaves over S . This is because, as described in ??, the graphic section must be evaluated on all points in the graphic space to generate the visual element corresponding to a data record at a single point $A(\tau(k)) = \rho(\xi^{-1}(k))$.

Since encoder functions are infinitely composable, as described in Equation 45, a new compositor function Q can be constructed by pre-composing v functions with the existing Q .

$$\Gamma(K, V) \xrightarrow{v} \Gamma(K, V') \xrightarrow{Q} \Gamma(S, H) \quad (43)$$

The composition in Equation 50 means that different measurable components can yield the same visual elements. The operators defined in subsection 4.3 can also act on compositors Q such that $Q_{a+b} = Q_a + Q_b$ and multiplied $Q_{a \times b} = Q_a Q_b$.

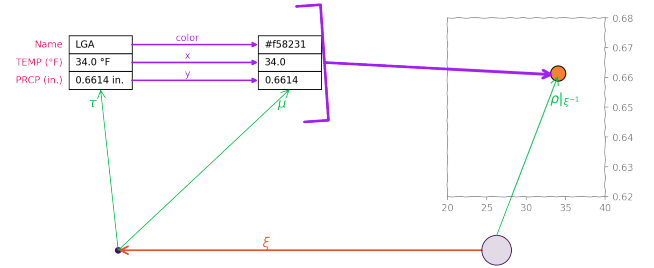


Figure 9: This simple Q assembles a circular visual element that is the color specified in $\mu(k)$ and is at the intersection specified in $\mu(k)$ much better labeling, include semantic labeling, make everything bigger

As shown in Figure 10, a set of v functions individually convert the values in the data record to visual components. Then the Q function combines these visual encodings to produce a graphic section ρ . When this section is evaluated on the graphic space associated with the data $\rho(\xi^{-1}(k))$, it produces a blue circular marker at the intersection of the x and y positions listed in μ . The composition rule in Equation 50 means that developers can implement Q as drawing circles or can implement a Q that draws arbitrary shapes, and then provide different v adapters, such as one that specifies that the shape is a circle.

5.3.1 Compositor Verification

An advantage of factoring out encoding and verification, as discussed in subsubsection 5.2.1, is that the responsibility of the compositor can be scoped to translating measurable components into

visual elements.

$$\begin{array}{ccc}
 \Gamma(K, V^a \times V^b) & \xrightarrow{Q_{ab}} & \text{Im}_{\mathcal{A}}(S, H) \\
 \downarrow \pi_a & & \downarrow M|_a \circ \delta_{ab} \\
 \Gamma(K, V^a) & \xrightarrow{\sim} & \text{Hom}(K, M^a) \\
 \uparrow \pi_a & & \uparrow M|_a \circ \delta_{ac} \\
 \Gamma(K, V^a \times V^c) & \xrightarrow{Q_{ac}} & \text{Im}_{\mathcal{A}}(S, H)
 \end{array} \quad (44)$$

As illustrated in Equation 51, a compositor is valid if there is an isomorphism between the actual outputted measurable visual component and the expected measurable component that is the input. One way of verifying that a compositor is consistent is by verifying that it passes through one encoding even while changing others. For example, when $Q_{ab} = Q_{ac}$ then the output should differ in the same measurable components as μ_{ab} and μ_{ac} .

A compositor function Q is equivariant if the renderer output changes in a way equivariant to the data transformation defined in subsection 4.2.1. This means that a change in base space $\hat{\phi}_E$ should have an equivalent change in visual element base space. This means that there should be no change in visual measurement

$$\mu(\hat{\phi}_K(k')) = \delta(Q(\mu)(\hat{\phi}_K(\xi^{-1}k))) = M_k \quad (45)$$

As discussed in Figure 8, the change in base space may induce a change in locations of measurements relative to each other in the output; this can be verified via checking that all the measurements have not changed relative to the original positions $M_k = M_{k'}$ and through separate measurable variables that encode holistic data properties, such as orientation or origin.

The compositor function is also expected to be equivariant with respect to changes in data and measurable components

$$\tilde{\phi}_V(\mu(k)) = \tilde{\phi}_M(Q(\mu(k))) \quad (46)$$

which means that any change to a measurable component input must have a measurably equivalent change in the output. As illustrated in Figure 8, the compositor Q is expected to assemble the measurable components such that base space changes, for example transposition, are reflected in the output; faithfully pass through equivariant measurable components, such as scaled colors; and ensure that both types of transformations, here scaling and transposition, are present in the final glyph.

5.4 Implementing the Artist

Since the artist is a family of functions in the homset between sheaves (??) the isomorphism allows for the specification of the transformation from data as a combination of functions over different spaces:

$$\begin{array}{ccccc}
 & & \mathcal{A}^K & & \\
 & \nearrow & & \searrow & \\
 \Gamma(K, E) & \xrightarrow{\gamma^K} & \Gamma(K, V) & \xrightarrow{Q^K} & \text{Im}_{\mathcal{A}}(K, \xi_*, H) \\
 & \searrow \mathcal{A} & \downarrow \xi^* & \searrow Q & \uparrow \xi_* \\
 & & \Gamma(S, \xi^* E) & \xrightarrow{\gamma^S} & \Gamma(S, \xi^* V) & \xrightarrow{Q^S} & \text{Im}_{\mathcal{A}}(S, H) \\
 & \nearrow \mathcal{A}^S & & \nearrow &
 \end{array} \quad (47)$$

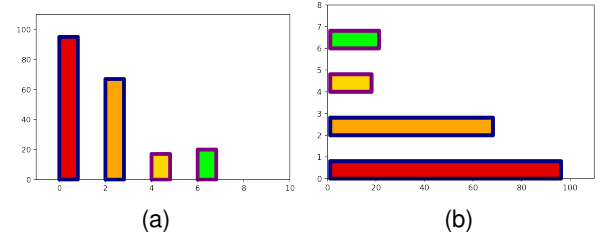
This means that an artist over data space $\mathcal{A}_K : \tau \mapsto \xi_* \rho$, an artist over graphic space $\text{vartist}_S : \xi^* \tau \mapsto \rho$, and an artist $\mathcal{A} : \tau \mapsto \rho$ are equivalent such that:

$$\begin{aligned}
 \tau(k) &= \xi^* \tau(s) \\
 \implies \mathcal{A}_K(\tau(k)) &= \mathcal{A}_S(\xi^* \tau(s)) = \mathcal{A}(\tau(k)) \\
 \implies \xi_* \rho(s) &= \rho(s)
 \end{aligned}$$

when $\xi(s) = k$. This equivalence allows a developer to connect transformations over data space, denoted with a subset K , with transformations over graphic space S , using ξ_* and ξ^* adaptors. This allows developers to for example connect transformers that transform data on a line to a color in data space, but build a line compositing function that dynamically resamples what is on screen in graphic space.

6 DISCUSSION: FEASIBILITY AS DESIGN SPEC

The framework specified in section 4 and section 5 describes how to build structure preserving visualization components, but it is left to the library developer to follow these guidelines when building and reusing components. In this section, we introduce a toy example of building an artist out of the components introduced in section 5 to illustrate how components that adhere to these specifications are maintainable, extendible, scalable, and support concurrency.



Specially, we introduce artists for building the graphical elements shown in 11 because it is a visualization type that allows us to demonstrate composability and multivariate data encoding. We build our visualization components by extending the Python visualization library Matplotlib's artist⁵[30, 29] to show that components using this model can be incorporated into existing visualization libraries iteratively. While the architecture specified in section 5 can be implemented fully functionally, we make use of objects to keep track of parameters passed into artists. In this toy example, the small composable components allow for more easily verifying that each component does its transformation correctly before assembling them into larger systems.

6.1 Bundle Inspired Data Containers

fruit	calories	juice
apple	95	True
orange	67	True
lemon	17	False
lime	20	False

We construct a toy dataset with a discrete K of 4 points and a fiber space of $F = \{\text{apple, orange, lemon}\} \times \mathbb{Z}^+ \times \{\text{True, False}\}$. We thinly wrap Table 2 in an object so that the common data interface function is that $\tau = \text{DataContainerObject.query}$.

⁵Matplotlib artists are our artist's namesake

```

1 class FruitFrameWrapper:
2     def query(self, data_bounds, sampling_rate):
3         # local sections are a list of
4         # {field: local_batch_of_values}
5         return local_sections

```

This interface provides a uniform way of accessing subsets of the data, which are local sections. The motivation for a common data interface is that it would allow the artist to talk to different common python data containers, such as numpy[22], pandas[45], xarray [26], and networkx[18]. Currently, data stored in these containers must be unpacked and converted into arrays and matrices in ways that either destroy or recreate the structure encoded in the container. For example a pandas data frame must be unpacked into its columns before it is sent into most artists and continuity is implicit in the columns being the same length rather than a tracked base space K . Because it is more efficient to work with the data in column order, we often project the fiber down into individual components. As shown in ??, we can verify that this projection is correct by checking that the values at the index are the same regardless of the level of decomposition.

6.2 Component Encoders

To encode the values in the dataset, we enforce equivariance by writing ν encoders that match the structure of the fields in the dataset. For example, the fruit column is a nominal measurement scale. Therefore we implement a position encoder that respects permutation $\hat{\phi}$ transformations. The most simple form of this ν is a python dictionary that returns an integer position, because Matplotlib's internal parameter space expects a numerical position type.

```

1 def position_encoder(val):
2     return {'apple': 0, 'orange': 2, 'lemon': 4, 'lime':
3         ↪ 6}[val]

```

As mentioned in Equation 45, the encoders can be composed up. For example, the compositor ν may need the position to be converted to screen coordinates. Here the screen coordinate ν is a method of a Matplotlib axes object; a Matplotlib axes is akin to a container artist that holds all information about the sub artists plotted within it.

```

1 def composite_x_transform(ax, nu):
2     return lambda x: ax.transData.transform(
3         (position_encoder(x), 0))[0]

```

This encoder returns a function that is `transData.transform` $\nu_{\text{transData}}$ composed with the position encoder ν_{position} and takes as input a record to be encoded. As with the position encoder, the `transData` encoder respects permutation transforms because it returns reals; therefore the composite encoder respects permutation transforms. In this model, developers implement ν encoders that are explicit about which ϕ_ν they support. Writing semantically correct encoders is also the responsibility of the developer and is not addressed in the model. For example `fruit_encoder = lambda x: {'apple': green, 'orange': 'yellow', 'lemon': 'red', 'lime': 'orange'}` is a valid color encoding with respect to permutation, but none of those colors are intuitive to the data. It is therefore left to the user, or domain specific library developer, to choose ν encoders that are appropriate for their data.

6.3 Graphic Compositors

After converting each record into an intermediate visual component μ , the set of visual records is passed into Q . Here the Q includes one last encoder, as illustrated in Equation 50, that assembles the independent visual components into a rectangle. This ν is inside the Q to hide that library preferred format from the user. It is called `qhat` to indicate that this is the A^K path in Equation 54. This means that the parameters are constructed in data space K and this function returns a pushed forward $\xi_*\rho$.

```

1 def qhat(position, width, length, floor, facecolor,
2     ↪ edgecolor, linewidth, linestyle):
3     box = box_nu(position, width, length, floor)
4     def fake_draw(render,
5         ↪ transform=mtransforms.IdentityTransform()):
6         for (bx, fc, ec, lw, ls) in zip(box, facecolor,
7             ↪ edgecolor, linewidth, linestyle):
8             gc = render.new_gc()
9             gc.set_foreground((ec.r, ec.g, ec.b, ec.a))
10            gc.set_dashes(*ls)
11            gc.set_linewidth(lw)
12            render.draw_path(gc=gc, path=bx,
13                ↪ transform=transform, rgbFace=(fc.r,
14                    ↪ fc.g, fc.b, fc.a))
15    return fake_draw

```

The function `fake_draw` is the analog of $\xi_*\rho$. This function builds the rendering spec through the renderer API, and this curried function is returned. The transform here is required for the code to run, but is set to identity meaning that this function directly uses the output of the position encoders. The curried `fake_draw` $\approx \xi_*\rho$ is evaluated using a renderer object. In our model, as shown in Equation 31, the renderer is supposed to take ρ as input such that `renderer( $\rho$ ) = visualization`, but here that would require an out of scope patching of the Matplotlib render objects.

One of the advantages of this model is that it allows for succinctly expressing the difference between two very similar visualizations, such as 11a and 11b. In this model, the horizontal bar is implemented as a composition of a ν that renames fields in μ_{barh} and the Q implementation for the horizontal bar.

```

1 def qhat(length, width, position, floor, facecolor,
2     ↪ edgecolor, linewidth, linestyle):
3     return Bar.qhat(**BarH.bar_nu(length, width, position,
4         ↪ floor, facecolor, edgecolor, linewidth, linestyle))

```

This composition is equivalent to $Q_{\text{barh}} = Q_{\text{bar}} \circ \nu_{\text{vtoh}}$, which is an example of Equation 50. These functions can be further added together, as described in subsection 4.3 to build more complex visualizations.

6.4 Integrating Components into an Existing Library

The ν and Q are wrapped in a container object that stores the $A = Q \circ \nu$ composition and a method for computing the μ .

```

1 class Bar:
2     def compose_with_nu(self, pfield, ffield,
3         nu, nu_inv):
4         # returns a new copy of the Bar artist
5         # with the additional nu that converts
6         # from a data (F) field value to a
7         # visual (P) field value

```

```

8     return new
9
10    def nu(self, tau_local): #draw
11        # uses the stored nus to convert data
12        # stored nus have F->P field info
13        return mus
14
15    @staticmethod
16    def qhat(position, width, length, floor, facecolor,
17             ↪ edgecolor, linewidth, linestyle):
18
19        return fake_draw

```

As shown in the draw method, generating a graphic section ρ is implemented as the composition of $qhat \approx Q$ and $nu \approx \nu$ applied to a local section of the sheaf $self.section.query \approx \tau^1$ such $draw \approx Q \circ \nu \circ \tau = A \circ \tau$. The ν and Q functions shown here are written such that they can generate a visual element given a local section $\tau|_{K_i}$ which can be as little or large as needed. This flexibility is a prerequisite for building scalable and streaming visualizations that may not have access to all the data.

This artist is then passed along to a shim artist that makes it compatible with existing Matplotlib objects (??). This shim object is hooked into the Matplotlib draw tree to produce the vertical bar chart in 11a. Using the Matplotlib artist framework means this new artist can be composed with existing artists, such as the ones that draw the axes and ticks. The example in this section is intentionally trivial to illustrate that the math to code translation is fairly straightforward and results in fairly self contained composable functions. A library applying these ideas, created by Thomas Caswell and Kyle Sunden, can be found at <https://github.com/matplotlib/data-prototype>. Further research could investigate building new systems using this model, specifically libraries for visualizing domain specific structured data and domain specific artists. More research could also explore applying this model to visualizing high dimensional data, particularly building artists that take as input distributed data and artists that are concurrent. Developing complex systems could also be an avenue to codify how interactive techniques are expressed in this framework.

7 CONCLUSION

The toy example presented in section 6 demonstrates that it is relatively straightforward to build working visualization library components using the construction described in section 5. Since these components are defined with single record inputs, they can be implemented such that they are concurrent. The cost of building a new function using these components is sometimes as small as renaming fields, meaning the new feature is relatively easy to maintain. These new components are also a lower maintenance burden because, by definition, they are designed in conjunction with tests that verify that they are equivariant. These new components are also compatible with the existing library architecture, allowing for a slow iterative transition to components built using this framework. The framework introduced in this paper is a marriage of the ways the graphic and data visualization communities approach visualization. The graphic community prioritizes ? how input is translated to output, which is encapsulated in the artist A . The data visualization community prioritizes the manner in which that input is encoded, which is encapsulated in the separation of stages $Q \circ \nu$. Formalizing that both views are equivalent $A = Q \circ \nu$ gives library developers the flexibility to build visualization components in the manner that makes more sense for the domain without having to sacrifice the equivariance of the translation.

REFERENCES

- [1] Cairographics.org. 4
- [2] M. J. Baker. EuclideanSpace: Maths - Sheaf.
- [3] A. Bartonicek, S. Urbanek, and P. Murrell. No More, No Less than Sum of Its Parts: Groups, Monoids, and the Algebra of Graphics, Statistics, and Interaction. 34(3):1063–1074. doi: 10.1080/10618600.2024.2429708 1
- [4] M. Bastian, S. Heymann, and M. Jacomy. Gephi: An open source software for exploring and manipulating networks. vol. 3 of *Proceedings of the International AAAI Conference on Web and Social Media*. 2
- [5] R. A. Becker and W. S. Cleveland. Brushing Scatterplots. 29(2):127–142. doi: 10.1080/00401706.1987.10488204 7
- [6] J. Bertin. II. The Properties of the graphic system. In *Semiology of Graphics*. ESRI Press. 8
- [7] J. Bertin. *Semiology of Graphics : Diagrams, Networks, Maps*. ESRI Press. 1, 12
- [8] Bokeh Development Team. *Bokeh: Python Library for Interactive Visualization*. 2
- [9] M. Bostock, V. Ogievetsky, and J. Heer. D³ Data-Driven Documents. 17(12):2301–2309. doi: 10.1109/TVCG.2011.185 2
- [10] T.-D. Bradley. What is a Functor? Definitions and Examples, Part 2.
- [11] T.-D. Bradley. What is a Natural Transformation? Definition and Examples. 7
- [12] T.-D. Bradley, J. Terilla, and T. Bryson. *Topology : A Categorical Approach*. MIT Press.
- [13] D. M. Butler and S. Bryson. Vector-Bundle Classes form Powerful Tool for Scientific Visualization. 6(6):576. doi: 10.1063/1.4823118 2
- [14] D. M. Butler and M. H. Pendley. A visualization model based on the mathematics of fiber bundles. 3(5):45. doi: 10.1063/1.168345 2
- [15] E. H. Chi. A taxonomy of visualization techniques using the data state reference model. In *IEEE Symposium on Information Visualization 2000. INFOVIS 2000. Proceedings*, pp. 69–75. doi: 10.1109/INFVIS.2000.885092 1
- [16] B. Geveci, W. Schroeder, A. Brown, and G. Wilson. VTK. 1:387–402. 2
- [17] R. W. Ghrist. *Elementary Applied Topology*, vol. 1. Createspace Seattle. 2, 5
- [18] A. A. Hagberg, D. A. Schult, and P. J. Swart. Exploring network structure, dynamics, and function using NetworkX. In G. Varoquaux, T. Vaught, and J. Millman, eds., *Proceedings of the 7th Python in Science Conference*, pp. 11–15. 2, 14
- [19] P. Hanrahan. VizQL: A language for query, analysis and visualization. In *Proceedings of the 2006 ACM SIGMOD International Conference on Management of Data*, SIGMOD ’06, p. 721. Association for Computing Machinery. doi: 10.1145/1142473.1142560 1
- [20] M. D. Hanwell, K. M. Martin, A. Chaudhary, and L. S. Avila. The Visualization Toolkit (VTK): Rewriting the rendering code for modern graphics cards. 1–2:9–12. doi: 10.1016/j.softx.2015.04.001 2
- [21] G. Harder and K. Diederich. *Lectures on Algebraic Geometry I: Sheaves, Cohomology of Sheaves, and Applications to Riemann Surfaces*. Aspects of Mathematics. Vieweg+Teubner Verlag. 6, 7
- [22] C. R. Harris, K. J. Millman, p. d. u. family=Walt, given=Stéfan J, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, et al. Array programming with NumPy. 585(7825):357–362. 14
- [23] A. Hatcher. *Algebraic Topology*. Cambridge University Press. 7
- [24] A. Head, A. Xie, and M. A. Hearst. Math augmentation: How authors enhance the readability of formulas using novel visual design practices. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*, CHI ’22, article no. 491. Association for Computing Machinery. doi: 10.1145/3491102.3501932 1
- [25] J. Heer and M. Agrawala. Software design patterns for information visualization. 12(5):853–860. doi: 10.1109/TVCG.2006.178 2
- [26] S. Hoyer and J. Hamman. Xarray: ND labeled arrays and datasets in Python. 5(1). 14
- [27] Z. Hu, J. Hughes, and M. Wang. How functional programming mattered. 2(3):349–370. doi: 10.1093/nsr/nwv042 1
- [28] J. Hughes. Why Functional Programming Matters. 32(2):98–107. doi: 10.1093/comjnl/32.2.98 1
- [29] J. Hunter and M. Droettboom. The Architecture of Open Source Applications (Volume 2): Matplotlib. 13

- [30] J. D. Hunter. Matplotlib: A 2D graphics environment. 9(3):90–95. doi: 10.1109/MCSE.2007.55 2, 13
- [31] G. Kindlmann and C. Scheidegger. An Algebraic Process for Visualization Design. 20(12):2181–2190. doi: 10.1109/TVCG.2014.2346325 1, 5
- [32] M. Krstajić and D. A. Keim. Visualization of streaming data: Observing change and context in information visualization techniques. In *2013 IEEE International Conference on Big Data*, pp. 41–47. doi: 10.1109/BigData.2013.6691713 8
- [33] N. Kruchten, A. M. McNutt, and M. J. McGuffin. Metrics-Based Evaluation and Comparison of Visualization Notations. 30(1):425–435. doi: 10.1109/TVCG.2023.3326907 2
- [34] W. A. Lea. A formalization of measurement scale forms. 1(1):81–105. doi: 10.1080/0022250X.1971.9989789 5
- [35] S. Mac Lane. *Categories for the Working Mathematician*, vol. 5. Springer Science & Business Media. 7
- [36] J. Mackinlay. Automatic Design of Graphical Presentations. 1
- [37] J. Mackinlay. Automating the design of graphical presentations of relational information. 5(2):110–141. doi: 10.1145/22949.22950 1, 2
- [38] J. Mackinlay, P. Hanrahan, and C. Stolte. Show me: Automatic presentation for visual analysis. 13(6):1137–1144. doi: 10.1109/TVCG.2007.70594 1
- [39] B. Milewski. *Category Theory for Programmers*. Bartosz Milewski. 7
- [40] J. R. Munkres. *Elements of Algebraic Topology*. Addison-Wesley. 3
- [41] T. Munzner. What: Data Abstraction. In *Visualization Analysis and Design*, AK Peters Visualization Series, pp. 20–40. A K Peters/CRC Press. doi: 10.1201/b17511-3 3
- [42] D. Nekrasovski, A. Bodnar, J. McGrenere, F. Guimbreti re, and T. Munzner. An evaluation of pan & zoom and rubber sheet navigation with and without an overview. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, CHI '06, pp. 11–20. Association for Computing Machinery. doi: 10.1145/1124772.1124775 8
- [43] Z. Qu and J. Hullman. Keeping multiple views consistent: Constraints, validations, and exceptions in visualization authoring. 24(1):468–477. doi: 10.1109/TVCG.2017.2744198 11
- [44] A. Quint. Scalable vector graphics. 10(3):99–102. doi: 10.1109/MMUL.2003.1218261 6
- [45] J. Reback, W. McKinney, jbrockmendel, p. u. family=Bossche, given=Joris Van, T. Augspurger, P. Cloud, gyoung, Sinhrks, A. Klein, M. Roeschke, S. Hawkins, J. Tratner, C. She, W. Ayd, T. Petersen, M. Garcia, J. Schendel, A. Hayden, MomIsBestFriend, V. Jancauskas, P. Battiston, S. Seabold, chris-b1, h-vetinari, S. Hoyer, W. Overmeire, alimcmaster1, K. Dong, C. Whelan, and M. Mehryar. Pandas-dev/pandas: Pandas 1.0.3. doi: 10.5281/zenodo.3715232 14
- [46] E. Riehl. *Category Theory in Context*. Dover Publications. 3, 5
- [47] A. Satyanarayan, K. Wongsuphasawat, and J. Heer. Declarative interaction design for data visualization. In *Proceedings of the 27th Annual ACM Symposium on User Interface Software and Technology*, pp. 669–678. ACM. doi: 10.1145/2642918.2647360 2, 6
- [48] C. A. Schneider, W. S. Rasband, and K. W. Elceiri. NIH Image to ImageJ: 25 years of image analysis. 9(7):671–675. doi: 10.1038/nmeth.2089 2
- [49] N. Sofroniew, T. Lambert, K. Evans, P. Winston, J. Nunez-Iglesias, G. Bokota, K. Yamauchi, A. C. Solak, ziyangczi, G. Buckley, M. Bussonnier, D. D. Pop, T. Tung, V. Hilsenstein, Hector, J. Freeman, P. Boone, alisterburt, A. R. Lowe, C. Gohlke, L. Royer, H. Har-Gil, M. Kittisopikul, S. Axelrod, kir0ul, A. Patil, A. McGovern, A. Rokem, Bryant, and G. Pe a-Castellanos. Napari/napari: 0.4.5rc1. doi: 10.5281/zenodo.4533308 2
- [50] E. Spanier. *Algebraic Topology*. McGraw-Hill Series in Higher Mathematics. Springer. 2
- [51] D. I. Spivak. Databases are categories. 2, 3
- [52] D. I. Spivak. Simplicial databases. doi: 10.48550/ARXIV.0904.2012 2, 3
- [53] S. S. Stevens. On the Theory of Scales of Measurement. 103(2684):677–680. 1, 5
- [54] C. Stolte, D. Tang, and P. Hanrahan. Polaris: A system for query, analysis, and visualization of multidimensional relational databases. 8(1):52–65. doi: 10.1109/2945.981851 1
- [55] S. Studies. Culturevis/imageplot. 2
- [56] M. A. Thomas. Mathematization, Not Measurement: A Critique of Stevens' Scales of Measurement. doi: 10.2139/ssrn.2412765 5
- [57] M. Tory and T. Moller. Rethinking visualization: A high-level taxonomy. In *IEEE Symposium on Information Visualization*, pp. 151–158. doi: 10.1109/INFVIS.2004.59 1
- [58] E. R. Tufte. *The Visual Display of Quantitative Information*. Graphics Press. 7
- [59] J. VanderPlas, B. Granger, J. Heer, D. Moritz, K. Wongsuphasawat, A. Satyanarayan, E. Lees, I. Timofeev, B. Welsh, and S. Sievert. Altair: Interactive Statistical Visualizations for Python. 3(32):1057. doi: 10.21105/joss.01057 2
- [60] P. Vickers, J. Faith, and N. Rossiter. Understanding Visualization: A Formal Approach Using Category Theory and Semiotics. 19(6):1048–1061. doi: 10.1109/TVCG.2012.294 2
- [61] H. Wickham. *Ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York. 2
- [62] H. Wickham and L. Stryjewski. 40 years of boxplots. 11
- [63] V. Wiels and S. Easterbrook. Management of evolving specifications using category theory. In *Proceedings 13th IEEE International Conference on Automated Software Engineering (Cat. No.98EX239)*, pp. 12–21. doi: 10.1109/ASE.1998.732561 2, 4
- [64] L. Wilkinson. *The Grammar of Graphics*. Statistics and Computing. Springer-Verlag New York, Inc., 2nd ed ed. 1, 2
- [65] K. Wongsuphasawat. Navigating the Wide World of Data Visualization Libraries. 1
- [66] K. Wongsuphasawat. Navigating the Wide World of Data Visualization Libraries (on the web). 1
- [67] B. A. Yorgey. Monoids: Theme and variations (functional pearl). 47(12):105–116. doi: 10.1145/2430532.2364520 2
- [68] C. Ziemkiewicz and R. Kosara. Embedding Information Visualization within Visual Representation. In Z. W. Ras and W. Ribarsky, eds., *Advances in Information and Intelligent Systems*, pp. 307–326. Springer Berlin Heidelberg. doi: 10.1007/978-3-642-04141-9_15 1, 6, 7, 8